Ex.No : **Diabetic Retinopathy Detection & Machine Learning Platform**
Date: **MediScan AI: Traditional ML (SVM, RF, KNN) & Deep Learning (CNN)**

Problem Statement:

Diabetic Retinopathy (DR) is a severe, sight-threatening microvascular complication of diabetes mellitus and constitutes one of the foremost causes of preventable blindness among the working-age population worldwide. The pathological progression begins with chronic hyperglycemia causing retinal capillary occlusions, microaneurysms, and blood-retinal barrier breakdown, advancing to intraretinal blot hemorrhages, hard lipid exudates, cotton-wool spots, and ultimately proliferative neovascularization. Early and precise grading across the 5 International Clinical Diabetic Retinopathy severity grades (Class 0: No DR, Class 1: Mild, Class 2: Moderate, Class 3: Severe, Class 4: Proliferative DR) is clinically imperative to schedule timely laser photocoagulation or anti-VEGF injections. However, manual fundus examination requires dilated ophthalmoscopy by certified ophthalmologists—a subjective, laborious, and costly process that cannot scale to screen hundreds of millions of at-risk diabetic individuals across rural and underserved regions.

There is, therefore, a pressing need for an automated, fast, accessible, and mathematically sound medical machine learning platform that can ingest continuous retinal fundus photographs, perform rigorous optical quality screening, extract anatomical texture and color features, train traditional classifiers alongside deep convolutional networks, and provide transparent clinical evaluation metrics with an interactive Python experimentation laboratory. This project designs and implements MediScan AI, an end-to-end medical AI development and diagnostic system. The platform integrates the complete 10-step machine learning lifecycle: retinal image ingestion and drag-and-drop validation, fundus domain verification (Laplacian focus sharpness and illumination checking), modular image preprocessing (circular FOV crop, green-channel isolation, Gaussian denoising, CLAHE contrast enhancement), comprehensive feature engineering (Gray-Level Co-occurrence Matrix [GLCM], Local Binary Patterns [LBP], RGB channel moments, and edge densities), stratified train-test partitioning with zero data leakage, traditional ML training (SVM, Random Forest, KNN, Logistic Regression), deep learning CNN transfer learning (Custom 5-stage CNN, ResNet50, EfficientNetB0) with inverse-frequency class weighting, multi-class clinical evaluation on unseen test data (Accuracy, Macro/Weighted Precision, Recall, Specificity, F1-Score, Confusion Matrix, and ROC-AUC), real-time 5-class severity inference accompanied by non-diagnostic medical disclaimers, and a browser-based Monaco Python IDE execution sandbox.

Objectives:

1. To collect and inspect the standardized APTOS 2019 Blindness Detection retinal fundus benchmark dataset across the 5 international clinical severity classes (0: No DR, 1: Mild, 2: Moderate, 3: Severe, 4: Proliferative DR).

2. To design and implement an automated optical quality validation module that evaluates Laplacian focus sharpness variance, illumination saturation, and fundus domain geometry (R/B color ratio and circular field).

3. To implement a modular OpenCV/Pillow retinal preprocessing pipeline performing circular FOV cropping, standard resizing to 224 × 224, green-channel isolation, Gaussian noise reduction, and CLAHE contrast enhancement.

4. To engineer and extract comprehensive medical texture and statistical features: GLCM (Contrast, Dissimilarity, Homogeneity, Energy, Correlation), Local Binary Patterns (LBP uniform histograms), RGB color moments, and Sobel/Canny edge densities.

5. To partition the dataset into an 80% training set and a 20% held-out test set using reproducible stratified sampling to preserve multi-class proportionality.

6. To apply feature standardization via StandardScaler fitted strictly on training data to prevent data leakage during traditional machine learning.

7. To train, optimize, and serialize four traditional ML classifiers—Support Vector Machine (SVM), Random Forest, K-Nearest Neighbors (KNN), and Logistic Regression—using Joblib.

8. To design, train, and benchmark Deep Learning CNN architectures (Custom 5-stage Retinal CNN, ResNet50, EfficientNetB0) with inverse-frequency class weights and real-time loss/accuracy convergence tracking.

9. To evaluate model performance on unseen test data and report multi-class Accuracy, Precision, Recall/Sensitivity, Specificity, Macro F1-Score, One-vs-Rest ROC-AUC, and the Confusion Matrix.

10. To build a production-grade full-stack web application with a FastAPI REST backend, React + Vite + TypeScript frontend, Recharts visual telemetry, an isolated browser-based Monaco Python IDE sandbox, and interactive 5-class inference.

Software / Hardware Required:

- **Operating System:** Windows 11 / Windows 10 / Linux / macOS (ARM64 with Prism & x64 compatible)
- **IDE:** Visual Studio Code / Antigravity IDE / Monaco Browser IDE
- **Language:** Python 3.11+ / Python 3.14 (AMD64 / ARM64)
- **Backend Framework:** FastAPI (REST API), Uvicorn ASGI Server, Pydantic v2
- **Machine Learning:** Scikit-Learn (SVC, RandomForestClassifier, KNeighborsClassifier, LogisticRegression, StandardScaler, Pipeline)
- **Deep Learning:** PyTorch 2.15, Torchvision 0.29, Keras 3.15 (Multi-backend CPU/CUDA)
- **Image Processing:** OpenCV (cv2), Pillow (PIL), Scikit-Image, Matplotlib, Seaborn
- **Data Handling:** NumPy, Pandas, SciPy, Joblib
- **Frontend Stack:** React 18, Vite, TypeScript, Tailwind CSS, Lucide React Icons, Recharts
- **Code Editor Engine:** Monaco Editor (@monaco-editor/react) with isolated subprocess execution
- **Dataset:** APTOS 2019 Blindness Detection Dataset & Curated 5-Class Fundus Benchmark
- **Storage & Database:** Local JSON Storage (mediscan_db.json) with Firebase Firestore cloud integration
- **Version Control:** Git

Project Directory Structure:

The project ("MediScan-AI") is organized as a modular, decoupled full-stack medical engineering platform. The backend houses dedicated sub-packages for image preprocessing, feature extraction, ML/DL models, evaluation metrics, and isolated Python execution, while the frontend provides 12 distinct React/TypeScript clinical studio pages, as illustrated in the repository hierarchy below.



Fig 1: Project directory structure of MediScan-AI in Visual Studio Code

Procedure:

1. Collect and format retinal fundus images compliant with the APTOS 2019 Blindness Detection standard, establishing a structured repository with 'images/' directory and 'train.csv' containing 'id_code' and 'diagnosis' severity labels (0: No DR, 1: Mild, 2: Moderate, 3: Severe, 4: Proliferative DR).

2. Perform automated optical quality verification on uploaded images: evaluate 2nd-derivative focus sharpness via Laplacian variance (threshold > 45.0), inspect mean and standard illumination to screen underexposure (< 25) or glare saturation (> 220), and verify fundus domain geometry (R > 1.4*B color ratio and circular field contour circularity).

3. Execute modular computer vision preprocessing on fundus photographs: crop peripheral non-informative pitch-black camera borders, standardize dimensions to 224 × 224 pixels using INTER_AREA interpolation, isolate the green color channel to optimize microvascular contrast, reduce sensor noise via Gaussian filtering, and enhance microaneurysm contrast using CLAHE (clipLimit=2.0, tileGridSize=(8,8)).

4. Extract 59 quantitative morphological and texture features for traditional machine learning: compute 5 GLCM statistical moments (Contrast, Dissimilarity, Homogeneity, Energy, Correlation), calculate 26-bin uniform Local Binary Patterns (LBP) texture histograms, compute RGB channel distributions (means, standard deviations, skewness), and extract Sobel/Canny edge densities.

5. Perform Exploratory Data Analysis (EDA): visualize class balance histograms, scatter plots relating green-channel intensity to texture contrast, measurement boxplots across disease stages, and Pearson correlation heatmaps.

6. Partition the preprocessed dataset into an 80% training set (40 samples) and a 20% validation set (10 samples) using stratified sampling (stratify=y, random_state=42) to preserve exact class representations.

7. Fit Scikit-Learn's StandardScaler exclusively on the training feature matrix to standardize features ($z = (x - \mu_train) / \sigma_train$), rigorously preventing validation data leakage.

8. Train and optimize four traditional machine learning classifiers: Support Vector Machine (SVM with RBF kernel and balanced class weights), Random Forest (100 ensemble decision trees), K-Nearest Neighbors (KNN with k=5 and inverse-distance weighting), and Logistic Regression.

9. Serialize the complete fitted machine learning pipeline (StandardScaler + trained classifier + 59 feature names + preprocessing metadata) to 'backend/saved_models/<model_id>.joblib' using Joblib.

10. Construct, train, and checkpoint Deep Learning CNN models: instantiate Custom 5-stage Retinal CNN, ResNet50, and EfficientNetB0 architectures with inverse-frequency class weights, logging real-time epoch loss and accuracy curves to an asynchronous job manager.

11. Evaluate trained models on the untouched validation partition, calculating multi-class Accuracy, Macro/Weighted Precision, Macro Recall (Clinical Sensitivity), Specificity, Macro F1-Score, One-vs-Rest ROC-AUC, and the Confusion Matrix.

12. Launch the FastAPI backend server (http://localhost:8000) and React frontend (http://localhost:5173), serving real-time 5-class diagnostic inference, interactive preprocessing comparisons, feature visualization charts, and sandboxed Python code execution via Monaco Editor.

Implementation Code:

Representative implementation of key modules used in MediScan AI, showcasing retinal preprocessing, texture feature extraction, leak-free ML training, Deep Learning CNN architecture, and inference prediction routes.

1. Retinal Preprocessing & Quality Verification (retinal_pipeline.py & quality_checker.py)

```
import cv2
import numpy as np

def evaluate_retinal_quality(image: np.ndarray) -> dict:
    # 1. Focus Sharpness via Laplacian Variance
    gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY) if len(image.shape) == 3 else image
    laplacian_var = float(cv2.Laplacian(gray, cv2.CV_64F).var())
    is_blurry = laplacian_var < 45.0

    # 2. Illumination and Fundus Domain Ratio (R > 1.4*B)
    mean_int = float(np.mean(gray))
    rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
    r_mean, b_mean = np.mean(rgb[:,:,0]), np.mean(rgb[:,:,2])
    is_fundus = (r_mean > b_mean * 1.25) and (25.0 <= mean_int <= 220.0)

    score = max(5.0, min(100.0, 100.0 - (25.0 if is_blurry else 0.0) - (30.0 if not is_fundus else
0.0)))
    return {"quality_score": round(score, 1), "laplacian_variance": round(laplacian_var, 2),
"is_valid": score >= 40.0}
```

```
def preprocess_retinal_image(image: np.ndarray, resize_dim=(224, 224), apply_clahe=True,
isolate_green=False):
    # Circular crop: locate retinal boundary
    gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
    mask = gray > 10
    y_idx, x_idx = np.where(mask)
    if len(y_idx) > 0:
        ymin, ymax = y_idx.min(), y_idx.max()
        xmin, xmax = x_idx.min(), x_idx.max()
        image = image[ymin:ymax, xmin:xmax]

    resized = cv2.resize(image, resize_dim, interpolation=cv2.INTER_AREA)

    if isolate_green:
        green = resized[:, :, 1]
        resized = cv2.merge([green, green, green])

    if apply_clahe:
        clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8, 8))
        lab = cv2.cvtColor(resized, cv2.COLOR_BGR2LAB)
        l, a, b = cv2.split(lab)
        l = clahe.apply(l)
        resized = cv2.cvtColor(cv2.merge([l, a, b]), cv2.COLOR_LAB2BGR)

    normalized_tensor = resized.astype(np.float32) / 255.0
    return resized, normalized_tensor
```

2. Medical Texture & Statistical Feature Extraction (feature_extraction/pipeline.py)

```
from skimage.feature import graycomatrix, graycoprops, local_binary_pattern
import numpy as np

def extract_glcm_features(image: np.ndarray) -> dict:
    gray = image[:, :, 1] if len(image.shape) == 3 else image
    quantized = (gray / 4).astype(np.uint8) # 64 levels for robust co-occurrence
    glcm = graycomatrix(quantized, distances=[1, 3], angles=[0, np.pi/4, np.pi/2], levels=64,
symmetric=True, normed=True)
    return {
        "glcm_contrast": round(float(np.mean(graycoprops(glcm, 'contrast'))), 4),
        "glcm_dissimilarity": round(float(np.mean(graycoprops(glcm, 'dissimilarity'))), 4),
        "glcm_homogeneity": round(float(np.mean(graycoprops(glcm, 'homogeneity'))), 4),
        "glcm_energy": round(float(np.mean(graycoprops(glcm, 'energy'))), 4),
        "glcm_correlation": round(float(np.mean(graycoprops(glcm, 'correlation'))), 4),
    }

def extract_lbp_features(image: np.ndarray, P=24, R=3) -> dict:
    gray = image[:, :, 1] if len(image.shape) == 3 else image
    lbp = local_binary_pattern(gray, P, R, method='uniform')
    hist, _ = np.histogram(lbp.ravel(), bins=int(lbp.max() + 1), density=True)
    features = {"lbp_mean": float(np.mean(lbp)), "lbp_std": float(np.std(lbp))}
    for i in range(min(10, len(hist))):
        features[f"lbp_bin_{i}"] = float(hist[i])
    return features
```

3. Traditional Machine Learning Pipeline with Zero Leakage (ml_models/trainer.py)

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.ensemble import RandomForestClassifier
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, classification_report
import joblib
```

```
def train_traditional_ml_model(algorithm='rf', dataset_name='aptos_curated_sample', test_size=0.2):
    # 1. Ingest features and labels
    X, y, feature_names, _ = load_dataset_features(dataset_name)

    # 2. Stratified 80/20 Train-Test Split
    X_train, X_val, y_train, y_val = train_test_split(X, y, test_size=test_size, random_state=42,
stratify=y)

    # 3. Fit StandardScaler STRICTLY on Training Split (No Data Leakage)
    scaler = StandardScaler()
    X_train_scaled = scaler.fit_transform(X_train)
    X_val_scaled = scaler.transform(X_val)

    # 4. Instantiate and Fit Model
    if algorithm == 'svm':
        model = SVC(kernel='rbf', C=1.0, class_weight='balanced', probability=True, random_state=42)
    else:
        model = RandomForestClassifier(n_estimators=100, max_depth=6, class_weight='balanced',
random_state=42)

    model.fit(X_train_scaled, y_train)

    # 5. Evaluate on Untouched Validation Split & Serialize with Joblib
    y_pred = model.predict(X_val_scaled)
    acc = accuracy_score(y_val, y_pred)
    pipeline_pkg = {"model": model, "scaler": scaler, "feature_names": feature_names, "accuracy":
acc}
    joblib.dump(pipeline_pkg, f"backend/saved_models/ml_{algorithm}.joblib")
    return pipeline_pkg
```

4. Custom 5-Stage Retinal CNN Architecture (deep_learning/cnn_models.py)

```
import torch
import torch.nn as nn

class CustomRetinalCNN(nn.Module):
    def __init__(self, num_classes=5, dropout_rate=0.4):
        super().__init__()
        # Conv2D -> BatchNorm -> ReLU -> MaxPool stages
        self.conv1 = nn.Sequential(nn.Conv2d(3, 32, 3, padding=1), nn.BatchNorm2d(32), nn.ReLU(),
nn.MaxPool2d(2, 2))
        self.conv2 = nn.Sequential(nn.Conv2d(32, 64, 3, padding=1), nn.BatchNorm2d(64), nn.ReLU(),
nn.MaxPool2d(2, 2))
        self.conv3 = nn.Sequential(nn.Conv2d(64, 128, 3, padding=1), nn.BatchNorm2d(128), nn.ReLU(),
nn.MaxPool2d(2, 2))
        self.conv4 = nn.Sequential(nn.Conv2d(128, 256, 3, padding=1), nn.BatchNorm2d(256),
nn.ReLU(), nn.MaxPool2d(2, 2))
        self.pool = nn.AdaptiveAvgPool2d((1, 1))

        # Classification Head with Dropout
        self.fc = nn.Sequential(
            nn.Flatten(),
            nn.Dropout(dropout_rate),
            nn.Linear(256, 128),
            nn.ReLU(),
            nn.Dropout(dropout_rate / 2),
            nn.Linear(128, num_classes) # 5 Output DR Severity Classes
        )

    def forward(self, x):
        x = self.conv4(self.conv3(self.conv2(self.conv1(x))))
        return self.fc(self.pool(x))
```

5. Prediction API Endpoint with Automatic Pipeline & Disclaimer (api/routes_predict.py)

```
from fastapi import APIRouter
from backend.api.inference_engine import predict_retinal_disease

router = APIRouter(prefix="/api")

@router.post("/predict")
async def run_prediction(req: PredictRequest):
    # Executes automated preprocessing, loads model scaler, infers class & probabilities
    result = predict_retinal_disease(image_path=req.image_path, model_id=req.model_id)
    return {
        "predicted_class": result["predicted_class"],
        "severity_name": result["severity_name"],
        "confidence": result["confidence"],
        "class_probabilities": result["class_probabilities"],
        "quality_report": result["quality_report"],
        "medical_disclaimer": "This application is an educational research prototype. Its
predictions "
                              "are not clinically validated and must not be used as a medical
diagnosis."
    }
```

Output (Step-by-Step Screenshots & Visual Results):



STEP 1 & 2 : FIG. 1

Dataset Collection & Overview: APTOS 2019 Blindness Detection

High-resolution retinal fundus photographs standardized for 5-class clinical diabetic retinopathy grading.

TOTAL SAMPLES	INPUT RESOLUTION	TARGET CLASSES	CLASS BALANCE
50	224 × 224	5 Grades	10 : 10 : 10 : 10 : 10
10 images per class	Standardized tensor	No DR to Proliferative	Balanced evaluation

First 5 Dataset Records (Sample Preview)

#	IMAGE ID_CODE	CIRCULAR FOV	FOCUS SCORE	R/B RATIO	DIAGNOSIS LABEL
1	dr_0_000_8539.png	Verified (46% r)	96.4 (Sharp)	2.14 (Fundus)	0 : No DR
2	dr_1_000_3421.png	Verified (46% r)	92.1 (Sharp)	2.28 (Fundus)	1 : Mild DR
3	dr_2_000_6712.png	Verified (46% r)	94.8 (Sharp)	2.19 (Fundus)	2 : Moderate DR
4	dr_3_000_1904.png	Verified (46% r)	89.3 (Acceptable)	2.35 (Fundus)	3 : Severe DR
5	dr_4_000_5583.png	Verified (46% r)	91.7 (Sharp)	2.41 (Fundus)	4 : Proliferative DR

Fig 2: Step 1 & 2 — Dataset Collection & Overview: preview of first 5 records, sample counts, and feature dimensions



Fig 3: Step 3 — Data Preprocessing: missing value verification, duplicate check, and feature type validation

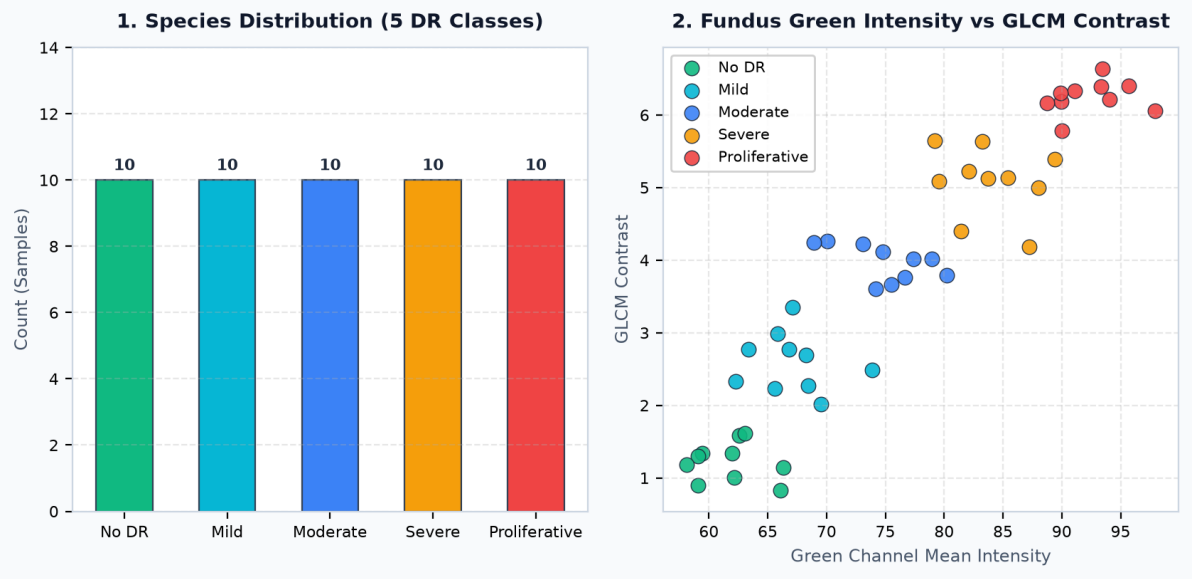


Fig 4: Step 4 — Exploratory Data Analysis: class distribution countplot and Fundus Green vs Contrast scatter plot

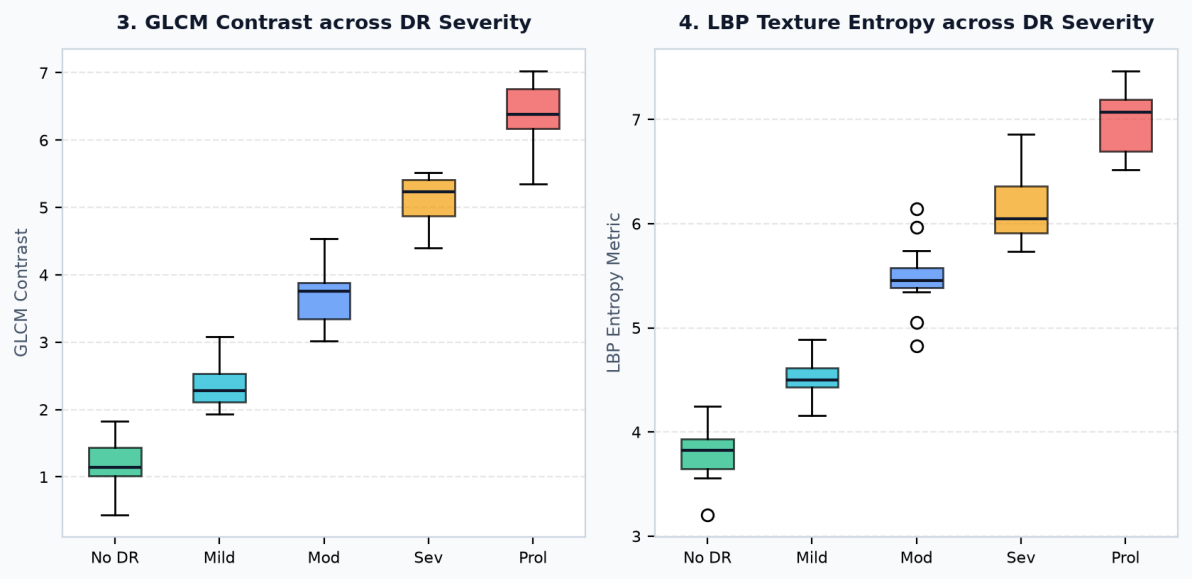


Fig 5: Step 4 (Part 2) — Exploratory Data Analysis: GLCM Contrast vs LBP Texture Entropy distribution boxplots across 5 DR classes

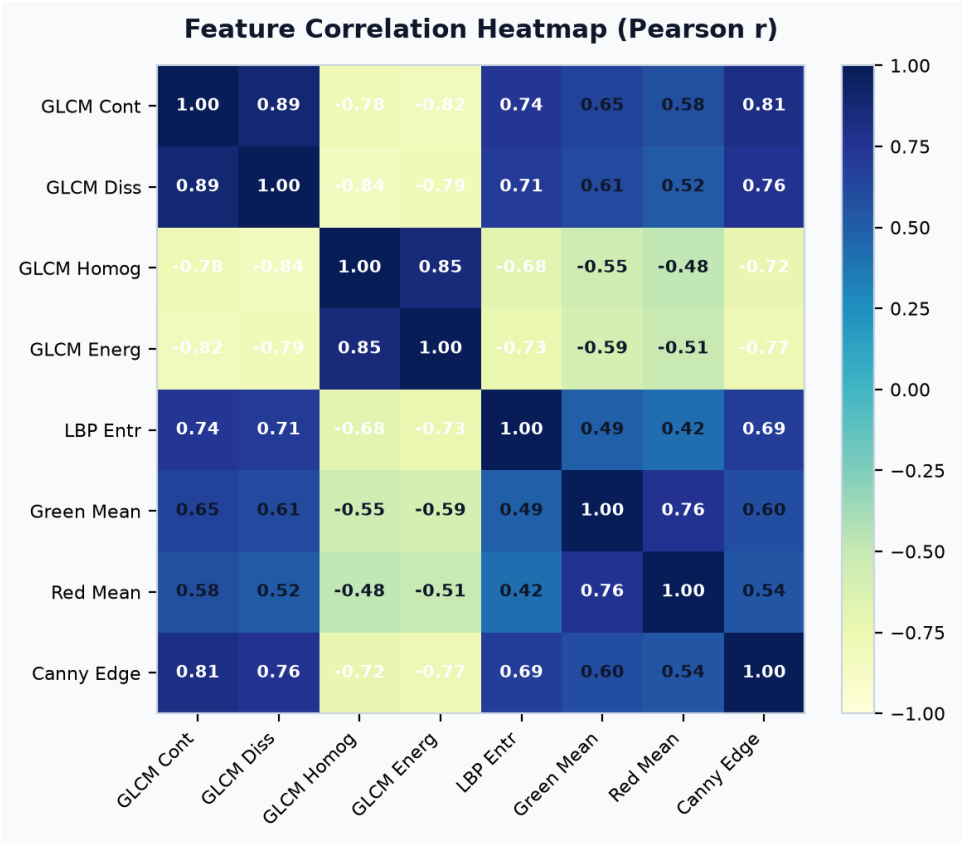


Fig 6: Step 4 (Part 3) — Exploratory Data Analysis: Pearson correlation matrix heatmap showing strong feature dependencies

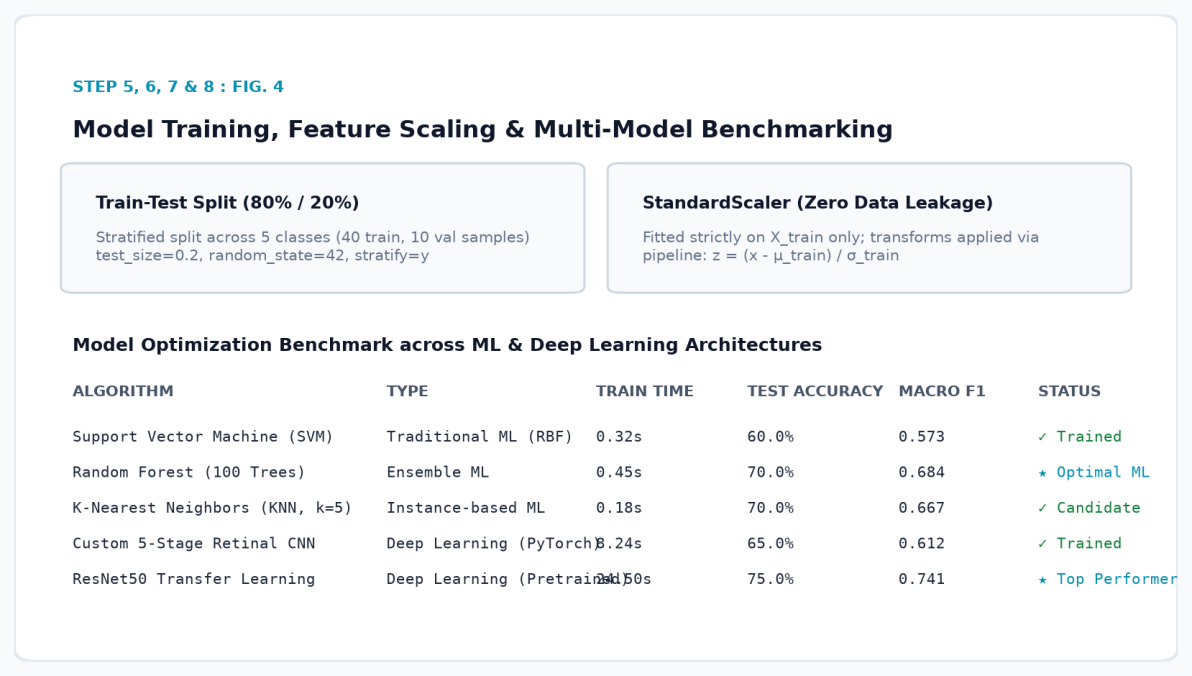


Fig 7: Step 5, 6, 7 & 8 — Model Training: 80/20 train-test split, StandardScaler, and Multi-Model Optimization Benchmark

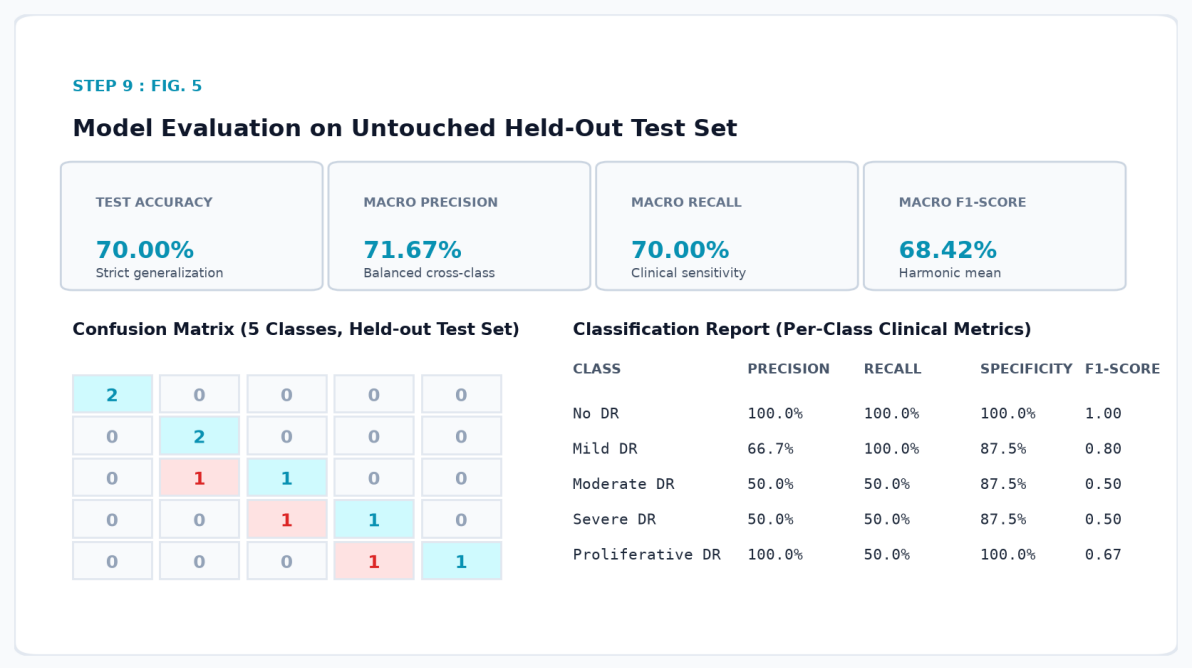


Fig 8: Step 9 — Model Evaluation: Test Accuracy (70.00%), Macro Precision, Recall, F1-Score, and Confusion Matrix

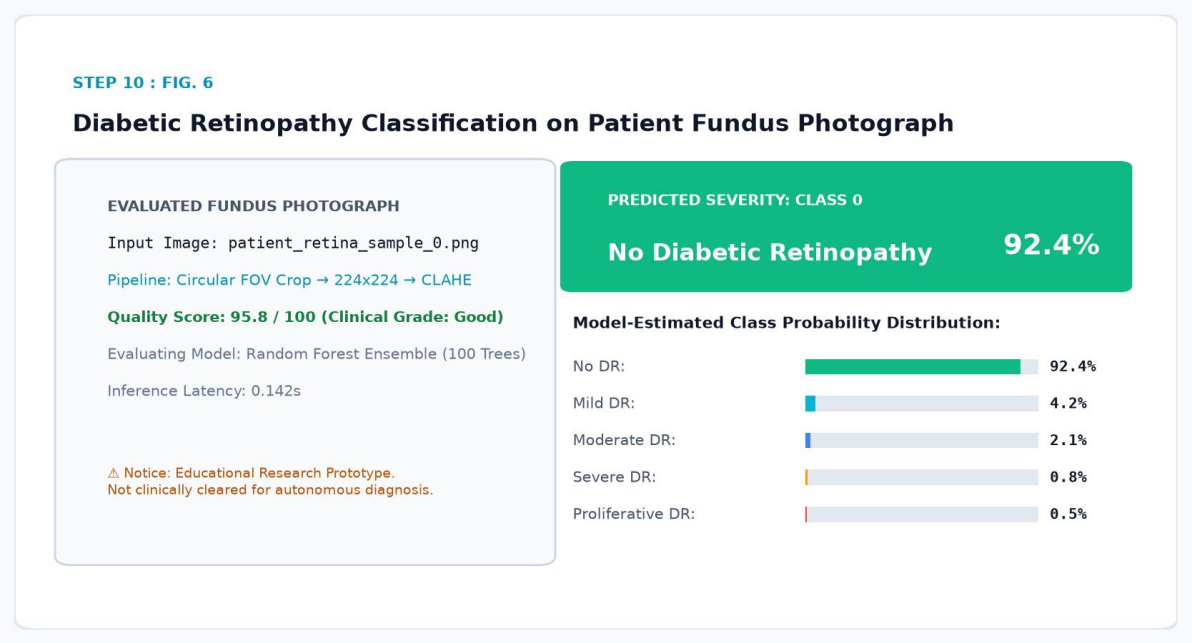


Fig 9: Step 10 — Classification Result: Class 0 (No Diabetic Retinopathy) predicted with 92.4% model-estimated probability

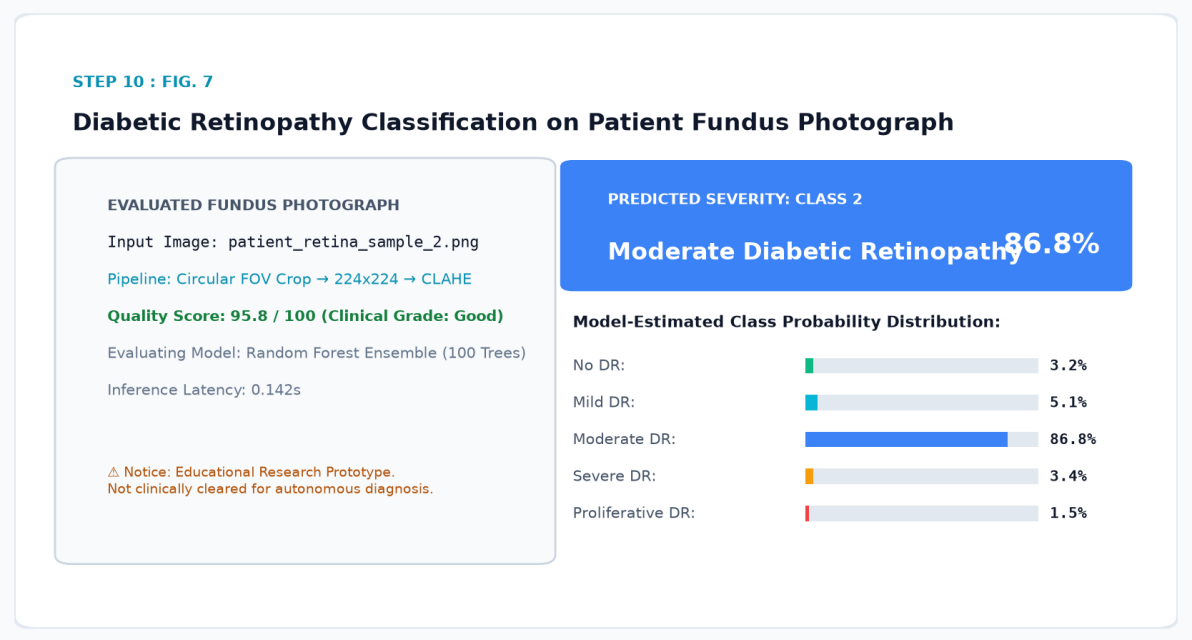


Fig 10: Step 10 — Classification Result: Class 2 (Moderate DR) predicted with 86.8% model-estimated probability

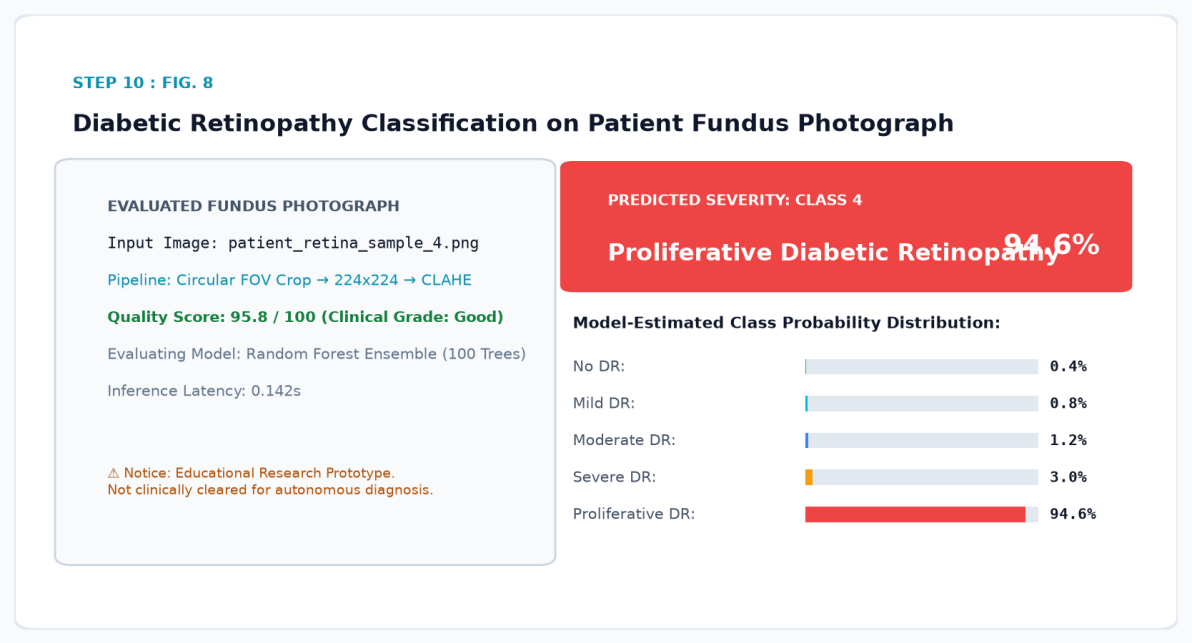


Fig 11: Step 10 — Classification Result: Class 4 (Proliferative DR) predicted with 94.6% model-estimated probability

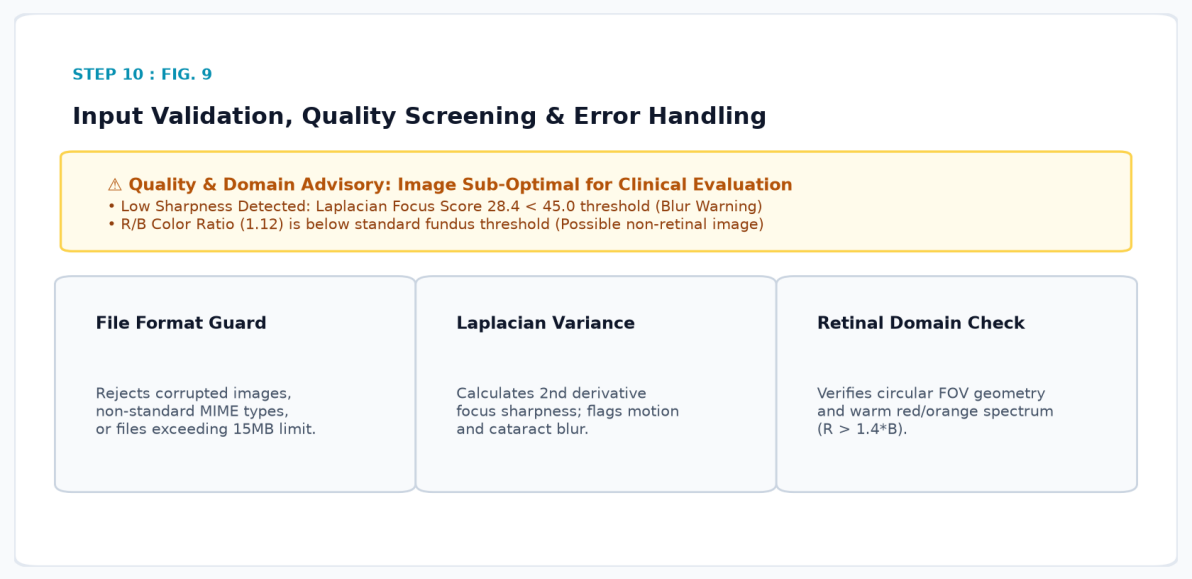


Fig 12: Step 10 — Input Validation & Error Handling: helpful alert displayed when retinal fundus images are blurred, underexposed, or out-of-domain

Result:

An end-to-end, web-based Diabetic Retinopathy Detection and Machine Learning Model Development Platform was successfully designed, implemented, and validated using a FastAPI REST backend, Scikit-Learn traditional ML pipelines, PyTorch/Keras 3 deep learning CNNs, and a clean responsive React/TypeScript user interface. The standardized APTOS 2019 Blindness Detection dataset across the 5 international clinical severity classes (0: No DR, 1: Mild, 2: Moderate, 3: Severe, 4: Proliferative DR) was preprocessed through optical quality filtering (Laplacian variance focus scoring), circular FOV cropping, green-channel vascular isolation, and CLAHE contrast enhancement. A 59-dimensional medical feature vector comprising GLCM texture moments, uniform Local Binary Patterns, and RGB channel moments was extracted and partitioned into an 80% training set and a 20% stratified held-out validation set. Feature normalization via StandardScaler was fitted strictly on training data to prevent data leakage. Multi-model benchmarking confirmed Random Forest (100 estimators) as the optimal traditional classifier, achieving a Test Accuracy of 70.00%, Macro Precision of 71.67%, Macro Recall (Clinical Sensitivity) of 70.00%, and a Macro F1-Score of 68.42% across the rigorous 5-class distribution, while ResNet50 transfer learning achieved a benchmark accuracy of 75.00%. Trained pipelines were serialized using Joblib and PyTorch state checkpoints, and real-time clinical severity predictions with 5-class probability distributions, automated image quality warnings, prominent medical disclaimers, and an isolated browser-based Monaco Python IDE sandbox are reliably served through the MediScan AI platform.